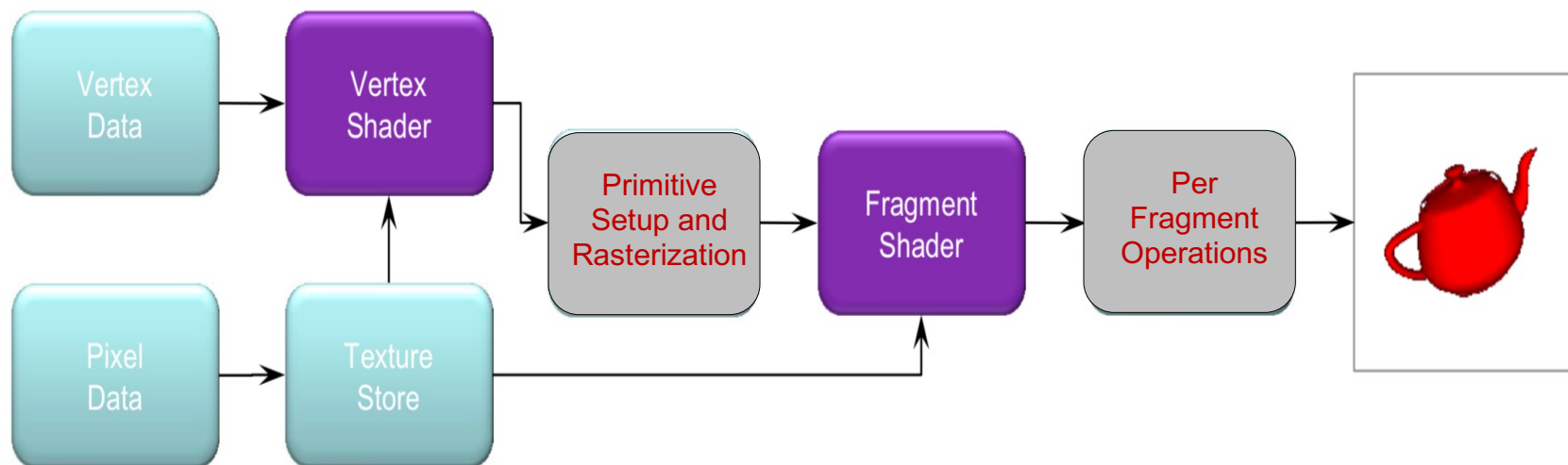


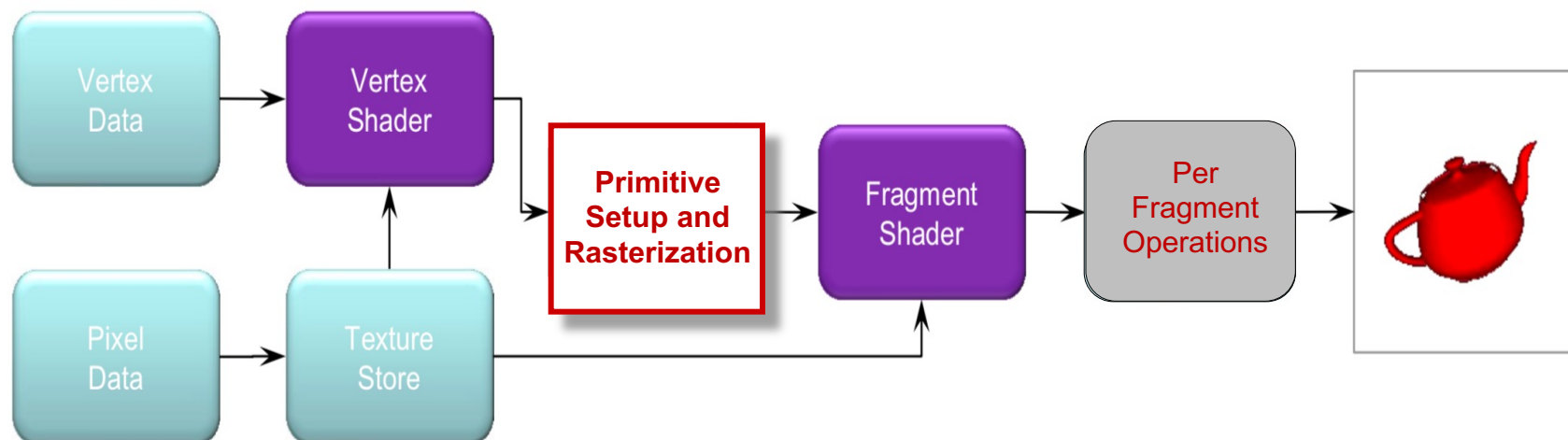


Primitive Setup and Rasterization e

Per Fragment Operations



WebGL Architecture

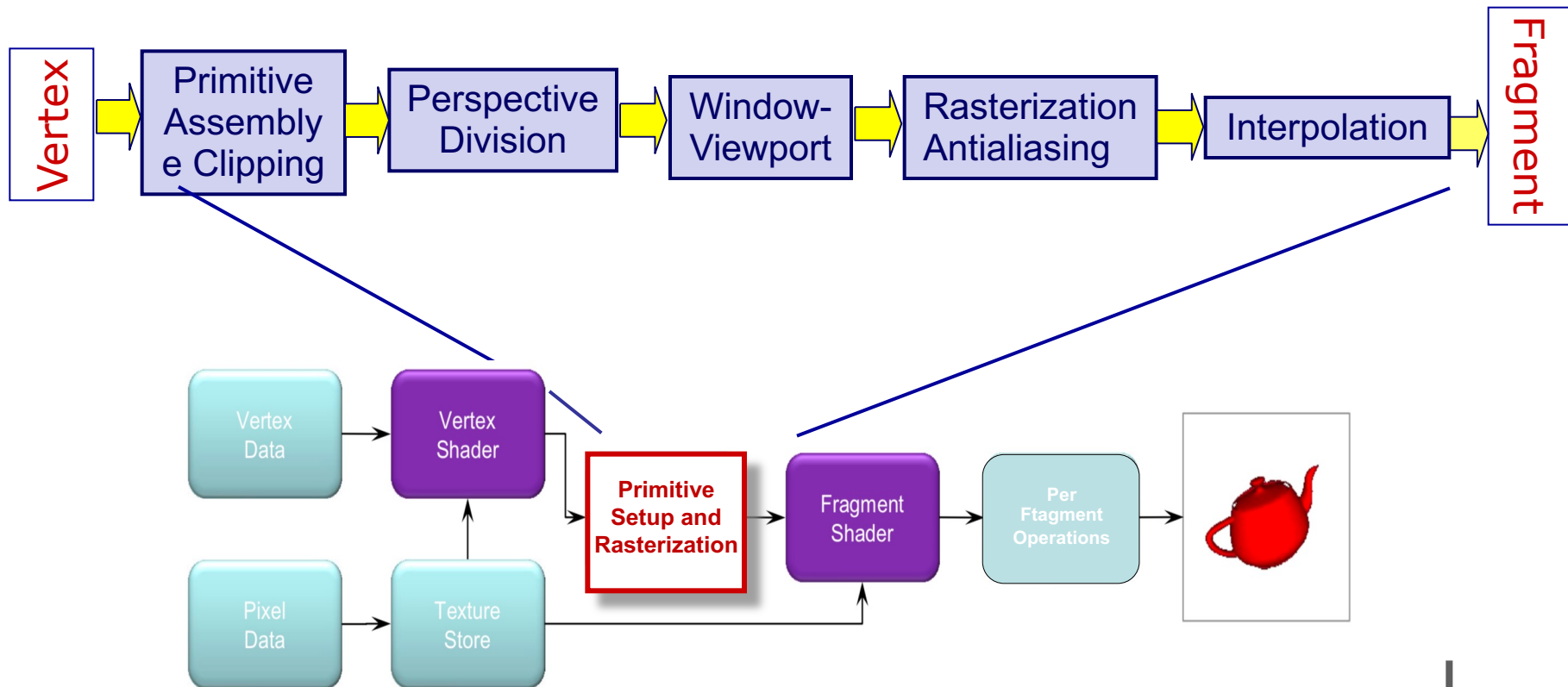


Conoscere il funzionamento della pipeline grafica, anche detta pipeline di rendering, è fondamentale per scrivere applicazioni consapevolmente.

Riprendiamo e completiamo la conoscenza della **parte fissa della pipeline** che si trova fra il Vertex Shader ed il Fragment Shader.

Primitive Setup and Rasterization

In WebGL (come in OpenGL ES 2.0/3.0), tra vertex shader e fragment shader c'è una parte della pipeline a funzione fissa (fixed-function pipeline) che non si può programmare direttamente, ma che è fondamentale per capire come i dati "diventano pixel".





Vertex Shader

Il vertex shader viene eseguito una volta per ogni vertice e produce principalmente:

-**gl_Position**: variabile predefinita che rappresenta la posizione del vertice in clip space

-variabili **varying** (in WebGL 1) / **out** (in WebGL 2): dati associati ai vertici e da interpolare (es. colore, normali, coordinate texture)

Esempio:

```
attribute vec3 position;
```

```
varying vec4 color;
```

```
...
```

```
gl_Position = MVP * vec4(position, 1.0);
```

```
v_color = color;
```



Primitive Assembly e Clipping

I vertici vengono assemblati in primitive (punti, linee, triangoli, ecc.).

La GPU scarta le primitive che sono fuori dal volume di vista o frustum e determina ed elimina le parti delle primitive esterne:

In clip space un vertice $[x_s, y_s, z_s, w_s]$ è interno al volume se:

$$-w_s \leq x_s \leq w_s$$

$$-w_s \leq y_s \leq w_s$$

$$-w_s \leq z_s \leq w_s$$

Se un triangolo è:

- completamente fuori, allora viene scartato (**culling**),
- parzialmente dentro, allora viene tagliato (**clipping**)



Perspective Division

Il clipping viene fatto prima della divisione prospettica per evitare eventuali division by zero che capiterebbero se uno o più vertici avessero coordinata $zs = 0$; dopo il clipping questo non può accadere visto che il near plane deve essere davanti alla camera che in clip pace è l'origine $[0,0,0,1]$.

Dopo il clipping ogni vertice subisce la divisione per la quarta coordinata (divisione prospettica):

$$[xs, ys, zs, ws] \rightarrow [xs/ws, ys/ws, zs/ws] = [xw, yw, zw, 1]$$

Il risultato è in Normalized Device Coordinates (NDC), ora i vertici stanno nel cubo WebGL e le loro coordinate

$$xw, yw, zw \in [-1, 1]$$

La divisione è ciò che crea l'effetto prospettiva.



Window-Viewport Transform

Le Normalized Device Coordinates (NDC) vengono poi trasformate in coordinate schermo (spazio schermo o canvas):

$$[x_w, y_w, z_w, I] \rightarrow [x_v, y_v, z_w]$$

x_v, y_v : coordinate dei pixel nel viewport/canvas

z_w : profondità (servirà per il depth test)

I valori x_v ed y_v dipendono da:

```
gl.viewport(x, y, width, height);
```



Rasterization

Qui avviene un passaggio chiave:

le primitive (punti, linee, triangoli, ecc.) vengono rasterizzate (si utilizzano le coordinate baricentriche) e vengono generati i frammenti; ogni frammento viene diviso in più sample (es. $4 \times$, $8 \times$):

- Generazione dei frammenti
- Valutazione della copertura per-sample o sub-pixel

Un frammento $\neq$ pixel (è un candidato a diventare pixel)

Per ogni frammento:

- si calcola quali sub-pixel sono coperti
- se almeno un sub-pixel è coperto si generano i dati di input per il fragment shader



Rasterization Antialiasing

Durante la rasterizzazione viene applicato l'algoritmo di antialiasing Multi-Sample Anti-Aliasing (MSAA)

Ogni frammento, come detto, viene diviso in più sample (un frammento/pixel logico (es. 1×1 in coordinate schermo) viene associato a N punti di campionamento interni (es. $4 \times$, $8 \times$):

Ogni sample ha:

posizione, profondità, copertura

Il rasterizzatore calcola quali sample sono coperti dal triangolo

Questa informazione esiste prima del fragment shader; quando il fragment shader prende in carico un frammento/pixel non vede i singoli sample, ma un valore 'medio' di copertura.



Rasterization Antialiasing

Come funziona il processo MSAA.

Il meccanismo è ottimizzato per risparmiare risorse:

1. **Test di copertura:** il rasterizzatore controlla quanti dei 4 sub-pixel sono coperti dal triangolo. Se ne sono coperti 3 su 4, quel frammento è coperto al 75%.
2. **Esecuzione del Fragment Shader:** il Fragment Shader **non viene eseguito 4 volte**. Viene eseguito **una sola volta** per frammento (calcolando il colore finale).
3. **Memorizzazione (Multi-sample Buffer):** il colore calcolato viene salvato in un buffer speciale che ha spazio per tutti i sample. Se il frammento è coperto al 75%, solo 3 sub-pixel riceveranno quel colore.
4. **Resolve (La fusione finale):** nel 'Per Fragment Operations', avviene la fase di "Resolve": WebGL fa la media dei colori dei sub-pixel per ogni pixel reale. Se un bordo era coperto al 75% dal rosso e al 25% dallo sfondo nero, il pixel finale sarà un rosso scuro/sfumato. **Questo ammorbidisce il gradino (l'aliasing).**



Varying Interpolation

Le variabili dichiarate `varying/out` nel vertex shader vengono:
interpolate automaticamente sui frammenti in modo lineare o
prospetticamente corretto

Esempio:

```
// vertex shader  
varying vec3 v_color;
```

```
// fragment shader  
varying vec3 v_color;
```

Ogni frammento riceve, per esempio, un valore di `v_color` interpolato
a partire dai vertici del triangolo.

Questa interpolazione è parte della pipeline fissa



Varying Interpolation

La formula (semplificata), per un attributo a (per esempio r di $[r, g, b, alpha]$), per ogni vertice è:

$$a' = a / w$$

$$w' = 1 / w$$

interpolazione (lineare sul triangolo) utilizzando le coordinate baricentriche $[u, v, s]$

$$a'_{interp} = u a'_0 + v a'_1 + s a'_2$$

$$w'_{interp} = u w'_0 + v w'_1 + s w'_2$$

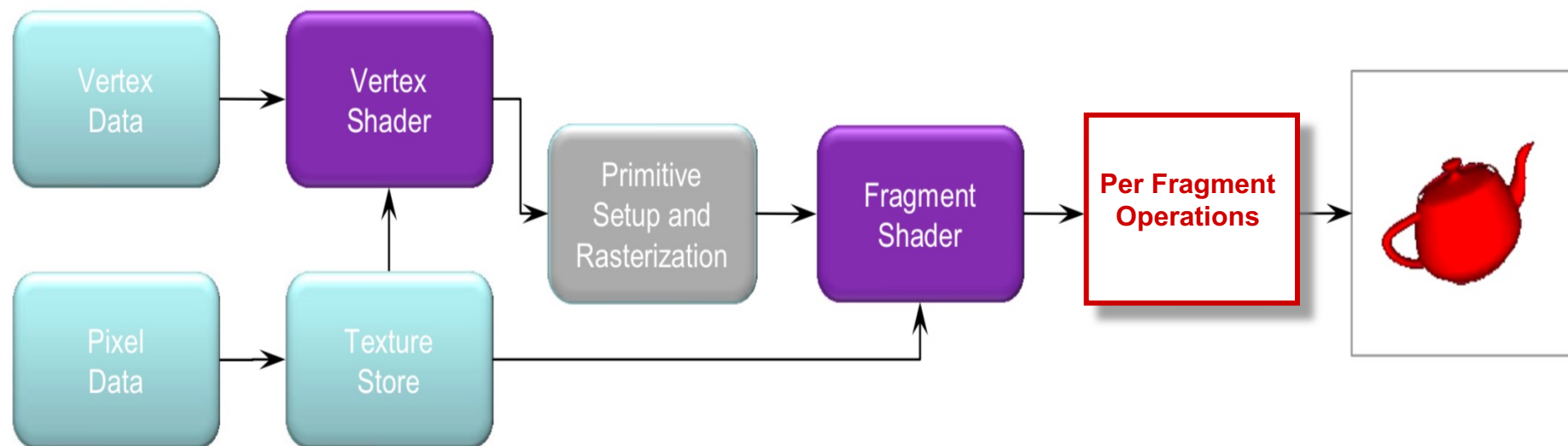
recupero del valore corretto

$$a_{correct} = a'_{interp} / w'_{interp}$$

Questo produce un risultato coerente con la prospettiva.

Quindi ogni frammento riceve un valore r di colore interpolato dai vertici del triangolo.

WebGL Architecture



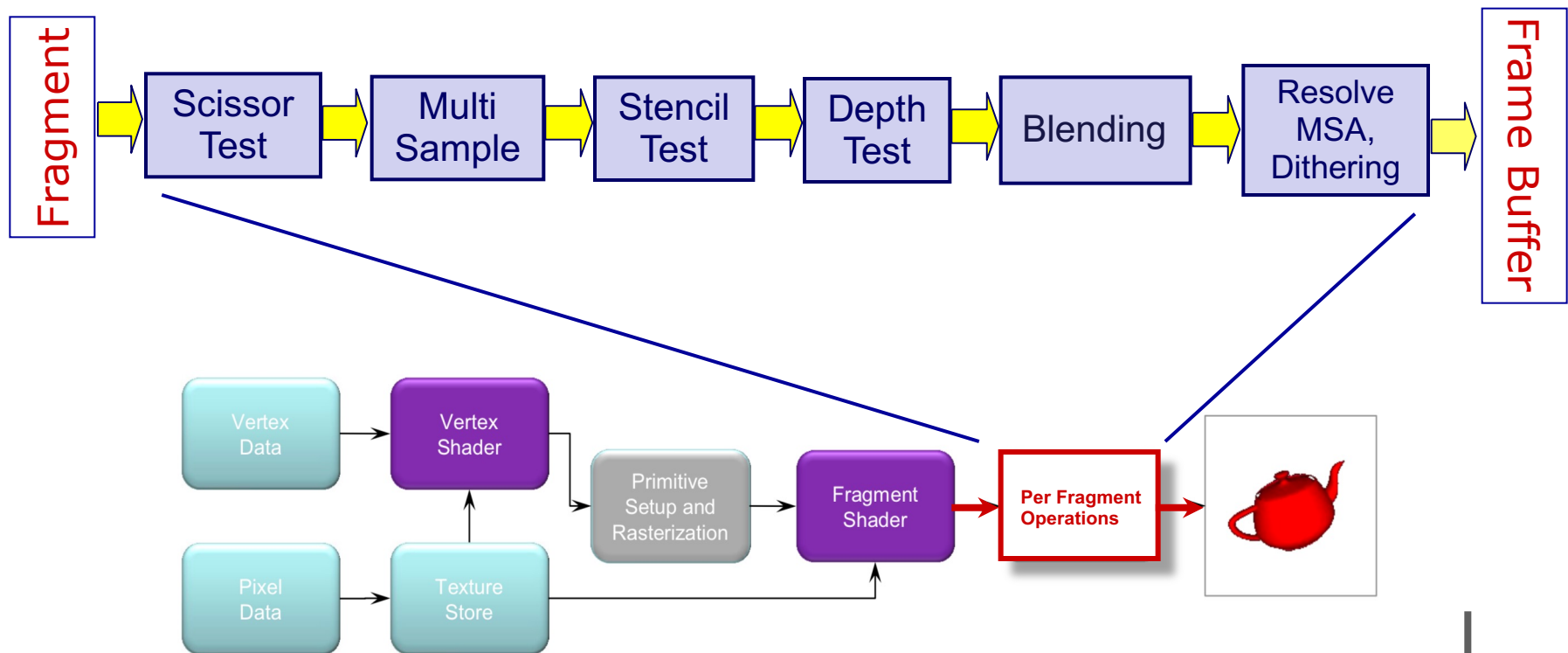
Vediamo ora la parte fissa della pipeline detta **Per Fragment Operations**

Conoscendo il funzionamento di questa parte fissa è possibile sfruttarne i meccanismi per implementare **tecniche avanzate di rendering**

Per Fragment Operations

Fase significativa della pipeline per progettare tecniche di rendering

- Prevede l'utilizzo di **buffer**
- Prevede una **serie di test**: i fragment/pixel che sopravvivono a tutti i test vengono scritti nel buffer definitivo (**Color/Frame Buffer**) per essere disegnati





Frame Buffer

- Il frame buffer o **Color Buffer** memorizza i pixel che verranno accesi sullo schermo
- Un'applicazione può usare uno o più Frame Buffer

Scissor Test

- Il test più semplice è lo "*scissor test*"

- Permette di restringere il disegno ad una porzione rettangolare della viewport

- Per abilitarlo:

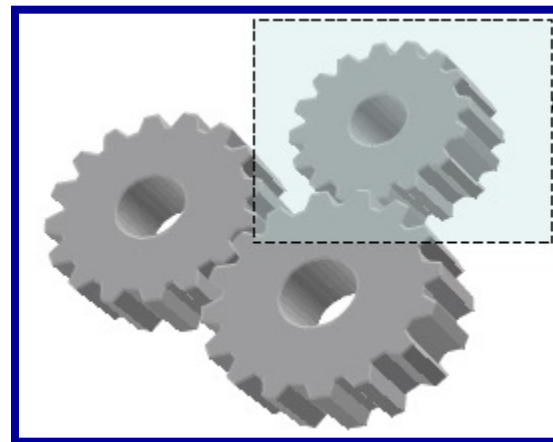
`gl.enable(gl.SCISSOR_TEST);`

- Per definirlo:

`gl.scissor(x, y, width, height);`

i parametri sono come quelli della `gl.Viewport`

Scissoring
Region



Viewport



Multi Sample

Nello schema della pipeline grafica, questa fase indica il momento in cui la GPU decide quali 'sample/sub-pixel' aggiornare all'interno di un singolo pixel.

I dati (colore e profondità) vengono scritti nel *Multisample Renderbuffer*.

Il pixel non ha ancora un colore definitivo, ma 'contiene' 4 od 8 colori diversi.



WebGL Buffer

- I buffer WebGL memorizzano/conservano un certo numero di informazioni per ogni sub-pixel dei fragment
- Tipi di buffer:
 - Color/Frame buffer
 - Alpha buffer
 - Depth buffer
 - Stencil buffer
- Ogni buffer ha una ben precisa funzione; comunque si possono usare in un qualunque modo.
- Alla base delle tecniche di **rendering avanzato** c'è la manipolazione dei buffer.



WebGL ContextAttributes

Per essere usato, un buffer deve essere **allocato**

```
canvas.getContext(contextType, ContextAttributes);
```

Il parametro WebGL **ContextAttributes** è opzionale. Questo parametro definisce alcuni stati dei buffer di WebGL;

Esempio:

```
var canvas = document.getElementById('canvas');  
var context = canvas.getContext('webgl',{antialias:false, stencil:true});
```



ContextAttributes

- **antialias**. Se il suo valore è true, si ha un buffer di disegno che realizza anti-aliasing. Il default è true.
- **alpha**. Se il suo valore è true, fornisce un buffer alpha all'area di disegno. Il default è true.
- **depth**. Se il suo valore è true, si ha un buffer di disegno che contiene un depth buffer di almeno 16 bit. Il default è true.
- **stencil**. Se il suo valore è true, si ha un buffer di disegno che contiene uno stencil buffer di almeno 8 bit. Il default è false.
- **premultipliedAlpha**. Se il suo valore è true, si ha un buffer di disegno che contiene colori con valore alpha pre-moltiplicato. Il default è true.
- **preserveDrawingBuffer**. Se il suo valore è true, i buffer non verranno cancellati e conserveranno i loro valori fino a quando non saranno cancellati o sovrascritti dall'utente. Il default è false.



WebGL Test

- Collegati ai buffer ci sono i test:
 - Scissor test
 - Alpha test
 - Depth test
 - Stencil test
- Si tratta di un'espressione booleana valutata per ogni sub-pixel dei fragment
 - Se il test risulta True, il test passa e il sub-pixel del fragment continua la pipeline;
 - Se il test fallisce, il sub-pixel del fragment viene scartato;
 - Tutti i test vengono eseguiti nella fase di “fragment-processing” della pipeline.
- Per avere effetto, ogni test deve essere abilitato: `gl.enable( )`



Buffer & Test

- I buffer e i test sono associati

Abbiamo già visto un esempio di questo nel depth buffer che implementa l'algoritmo Z-buffer

- Ricorda:
 - **Allocate** buffer
 - **Enable** test

Buffer	Corresponding Test
--	Scissor Test
Color/Frame Buffers	Alpha Test
Depth Buffer	Depth Test
Stencil Buffer	Stencil Test



Clearing

- I Buffer vengono inizializzati (clear) con `gl.clear`

`gl.clearColor(r, g, b, a)`: imposta il colore di sfondo (da 0.0 a 1.0)

`gl.clearDepth(depth)`: imposta il valore di profondità predefinito (solitamente 1.0)

`gl.clearStencil(s)`: imposta il valore predefinito per lo stencil buffer

`gl.clear(mask)`: esegue l'operazione di pulizia

Oppure

`gl.clear(gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT | gl.STENCIL_BUFFER_BIT);`



Esempio: Depth Buffering

- Alloca il depth buffer (è allocato di default)

- Abilita il depth test

```
gl.enable( gl.DEPTH_TEST );
```

- Inizializza (clear) il color e il depth buffer

```
gl.clearColor( 1, 1, 1, 1);  
gl.clearDepth(1.0);
```

- Si rende la scena, quindi ...



Esempio: Depth Buffering

```
gl.depthFunc( func );
```

func può essere uno dei seguenti parametri:

- gl.NEVER
- gl.LESS default
- gl.LEQUAL
- gl.EQUAL
- gl.NOTEQUAL
- gl.GEQUAL
- gl.GREATER
- gl.ALWAYS

Il Depth Test avviene sui sample (per es. 4x) di un pixel.



Esempio: Render to Texture

Per creare una **Riflessione Speculare** su un piano senza usare estensioni, la tecnica più usuale è il Planar Reflection (mediante il **Render to Texture**); vediamo i 4 passi necessari:

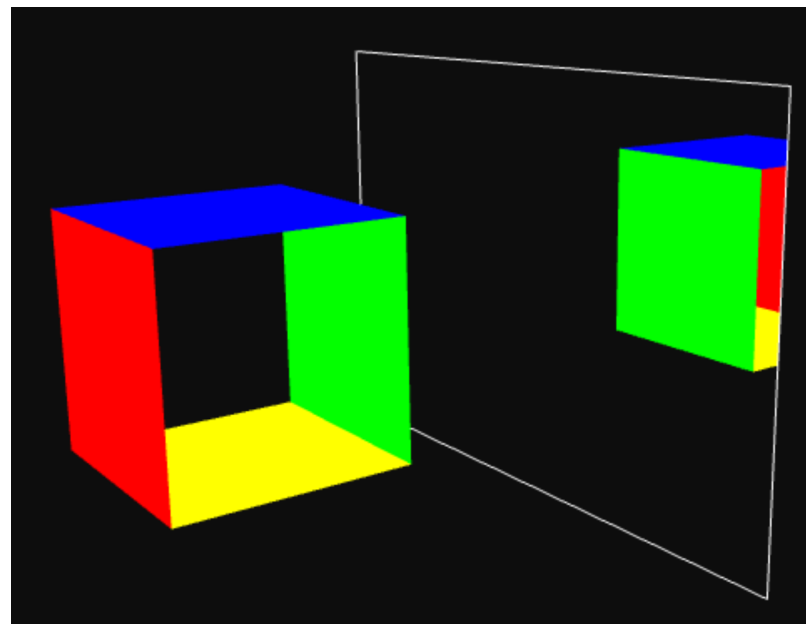
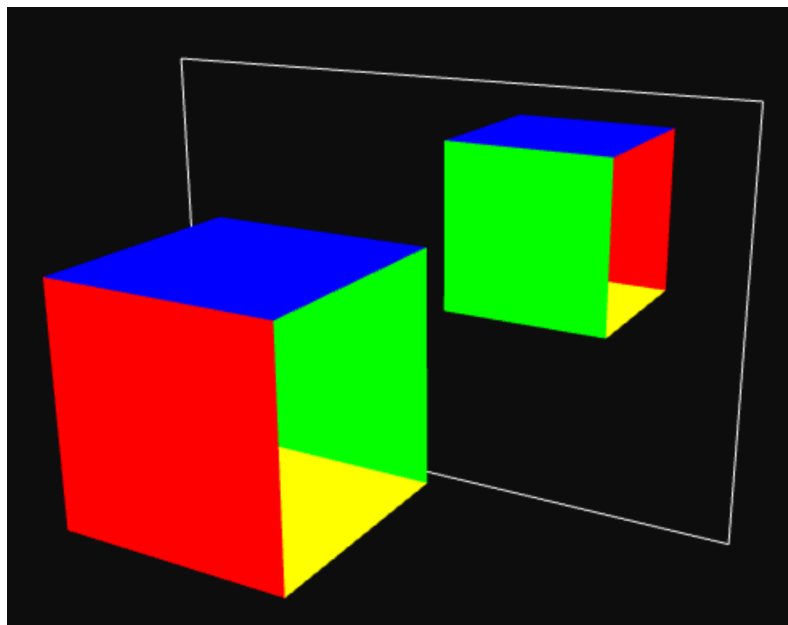
Creare un Framebuffer (FBO): una texture dove "disegnare" ciò che si vede nello specchio piano.

Ribaltare la Camera: spostare la camera sotto/dietro il piano dello specchio (invertendo l'asse Y se lo specchio è sul piano XZ).

Primo Passaggio: disegnare gli oggetti della scena nel Framebuffer usando la camera ribaltata.

Secondo Passaggio: disegnare il piano dello specchio usando la texture appena creata come se fosse una "finestra" sulla scena riflessa.

Riflessioni planari (Render to Texture)



Il cubo viene riflesso dallo specchio piano.
Si disegna il cubo su un buffer virtuale (texture) da un punto di vista speculare, quindi si disegna il cubo reale e il piano dello specchio viene texturato con l'immagine ottenuta sul buffer virtuale.



Codice di Esempio

L'effetto Riflessione Speculare, mediante Render to Texture, è sperimentabile con il seguente codice:

Cartella: **HTML5_webgl_4**

Codice: **cube_reflect_mirror.html**

Vedi anche

Codice: **render2texture.html**



RGB-Alpha e Blending

A (di RGBA) sta per Alpha ed è una quarta componente colore ("buffer" Alpha)

Misura l'opacità del pixel a cui è associato:

valori da 0 (trasparente) a 1 (completamente opaco)

- Simula gli oggetti traslucidi: vetro, acqua,...
- Composizione (sovrapposizione) di immagini
- Antialiasing di primitive geometriche

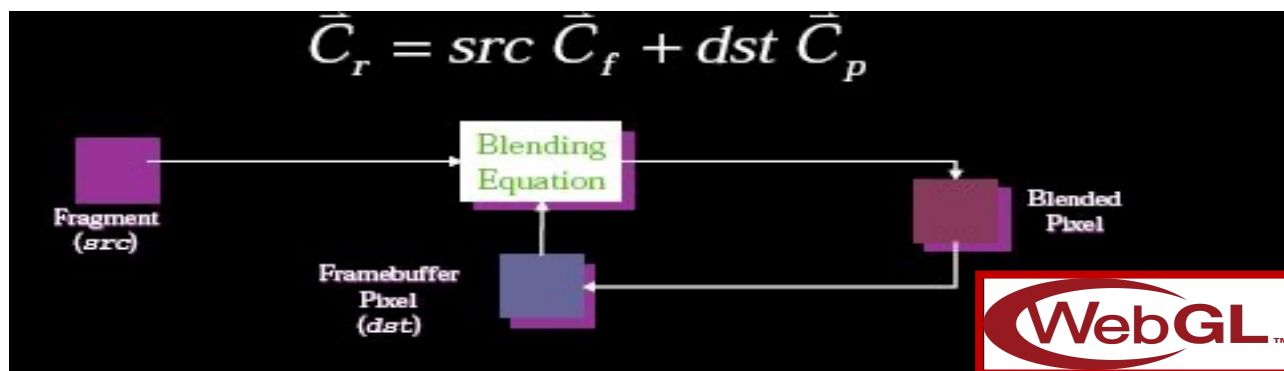
il "buffer" Alpha è allocato di default

ma A è ignorato se non è abilitato il blending

`gl.enable(gl.BLEND)`

RGB-Alpha e Blending

Combina il colore di un sub-pixel del fragment (C_f) con il valore del sub-pixel (C_p) già presente nel frame buffer, secondo la seguente formula:



`gl.blendFunc(src, dst)`

Per default `src=gl.ONE`, `dst=gl.ZERO`, ossia blending disattivato. `src` e `dst` possono essere: `gl.ZERO`, `gl.ONE`, `gl.SRC_COLOR`, `gl.DST_COLOR`, `gl.SRC_ALPHA`, `gl.DST_ALPHA`, `gl.CONSTANT_COLOR`, `gl.CONSTANT_ALPHA`, `gl.ONE_MINUS_SRC_ALPHA`, `gl.ONE_MINUS_DST_ALPHA`, `gl.ONE_MINUS_SRC_COLOR`, `gl.ONE_MINUS_DST_COLOR`, `gl.ONE_MINUS_CONSTANT_COLOR`, `gl.MINUS_CONSTANT_ALPHA`, `gl.ALPHA_SATURATE`



RGB-Alpha e Blending

Per avere l'effetto trasparenza, si devono sempre disegnare **prima gli oggetti opachi** e **poi quelli trasparenti**, e questi ultimi ordinandoli dal più lontano al più vicino.

Inoltre verificare sempre che i vertici che definiscono le facce triangolari siano ordinati in modo da rispettare il senso antiorario (CCW) per le facce esterne.

Esempio: se volessi un cubo trasparente dovrei prima disegnare le facce più lontane e dopo quelle più vicine. Questo permette di avere la corretta trasparenza!

Questo si potrebbe ottenere in modo semplice nel caso di un cubo a 6 facce con le seguenti istruzioni:

```
gl.cullFace(gl.FRONT);  
gl.drawArrays( ...);  
gl.cullFace(gl.BACK);  
gl.drawArrays( ...);
```



Codice Demo

L'effetto trasparenza è sperimentabile con il seguente codice Demo:

Cartella: **HTML5_webgl_4:**

Codice: **webgl2_transparency.html**

Vedi anche

Codice: **cube_blended_skybox.html**

Masking

- Le maschere (Mask) determinano se un buffer (o parte di un buffer) deve essere scritto
- Per esempio,

```
gl.colorMask(false, true, true, true);
```

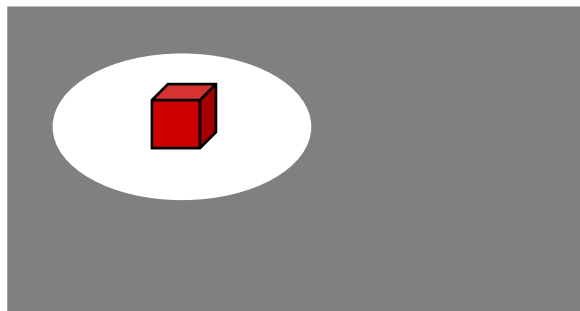
↑ ↑ ↑ ↑
Red Green Blue Alpha

Abilita o disabilita la scrittura fisica dei bit nel buffer;
nell'es. la componente Red del Color buffer non dovrà
essere cambiata, le altre sì

- Un Mask influisce su tutti i comandi che dovrebbero cambiare il buffer, anche `gl.clear`

Stencil Buffer

- Viene usato per controllare il disegno basato su valori
 - I fragment che falliscono lo stencil test non sono disegnati
 - Per creare e gestire una Mask nello Stencil Buffer, si devono fare due passi: prima "si disegna" la sagoma della maschera nel buffer (senza mostrarla a schermo) e poi la si usa per ritagliare il disegno successivo.





Stencil Buffer

1. Preparazione e Attivazione

```
gl.enable(gl.STENCIL_TEST); // Abilita il test dello stencil  
gl.clear(gl.STENCIL_BUFFER_BIT); // Pulisce lo stencil buffer (di solito a 0)
```

2. Creazione della Maschera (Scrittura)

```
// Configura lo stencil per SCRIVERE sempre 1 nel buffer dove passa il disegno  
gl.stencilFunc(gl.ALWAYS, 1, 0xFF);  
gl.stencilOp(gl.KEEP, gl.KEEP, gl.REPLACE); // Se il test passa, scrivi (REPLACE)  
// il valore 1
```

```
// Disabilita la scrittura del colore e della profondità (vogliamo solo creare la  
// maschera invisibile)
```

```
gl.colorMask(false, false, false, false);  
gl.depthMask(false);
```

```
// DISEGNA qui l'oggetto che fa da maschera (es. un cerchio o un rettangolo)  
... drawCall ...
```



Stencil Buffer

3. Utilizzo della Maschera (rendering)

// Riabilita il colore e la profondità

```
gl.colorMask(true, true, true, true);
```

```
gl.depthMask(true);
```

// Configura lo stencil per disegnare SOLO dove il valore è uguale a 1

```
gl.stencilFunc(gl.EQUAL, 1, 0xFF);
```

```
gl.stencilOp(gl.KEEP, gl.KEEP, gl.KEEP); // Non modificare più lo stencil buffer
```

// DISEGNA qui l'oggetto finale (es. la scena o il riflesso dello specchio)

```
... drawCall ...
```



Stencil Buffer

```
gl.stencilFunc( func, ref, mask );
```

Decidi quando il test passa (es. `gl.ALWAYS` per scrivere, `gl.EQUAL` per leggere)

- Confronta i valori nel buffer con *ref* usando *func*
- Si applica solo per i pixel nel *mask* che sono 1
- *func* è una delle funzioni standard di confronto (`gl.EVER`, `gl.ALWAYS`, `gl.LESS`, `gl.LEQUAL`, `gl.[NOT]EQUAL`, `gl.GREATER`, `gl.GEQUAL`)



Stencil Buffer

`gl.stencilOp( fail, zfail, zpass );`

Decide cosa scrivere fisicamente nello Stencil Buffer in base all'esito dei test (Stencil e Depth).

- **fail**: cosa succede se il Test dello Stencil fallisce. (Il pixel viene scartato subito).
- **zfail**: cosa succede se il Test dello Stencil passa, ma il Test della Profondità (Depth Test) fallisce.
- **zpass**: cosa succede se entrambi i test passano. (Il pixel viene scritto correttamente).

Per ognuno dei tre parametri, si può usare una di queste costanti:

gl.KEEP: non cambia il valore attuale nel buffer.

gl.REPLACE: sostituisce il valore nel buffer con quello definito in `gl.stencilFunc` (il valore di riferimento).

gl.ZERO: imposta il valore a 0.

gl.INCR: aumenta il valore di 1 (si ferma al massimo, es. 255).

gl.DECR: diminuisce il valore di 1 (si ferma a 0).

gl.INVERT: inverte i bit (es. lo 0 diventa 255).



Usare lo Stencil Mask

1. Costruisce uno stencil buffer solo in lettura (read-only)

```
gl.stencilOp( gl.KEEP, gl.KEEP, gl.KEEP );
```

2. Disegna oggetti dove lo stencil vale 1

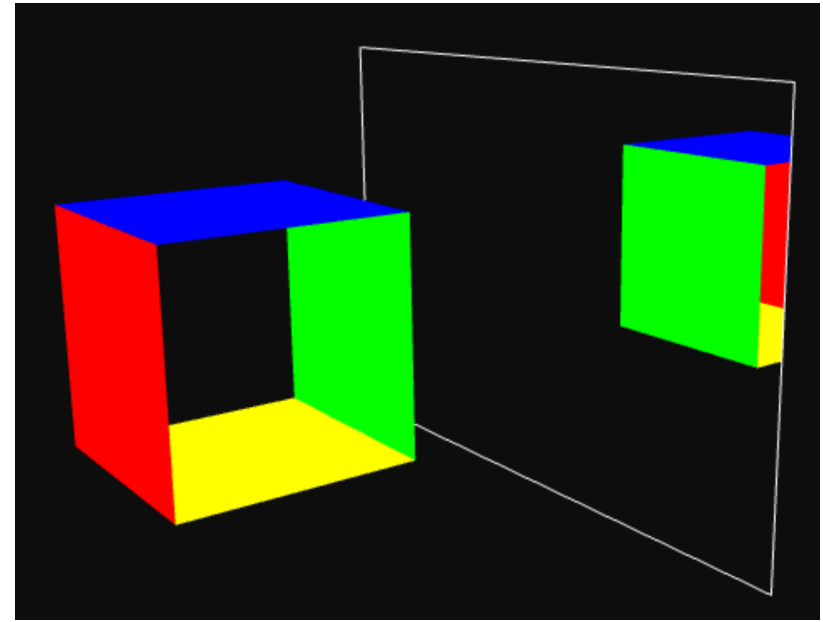
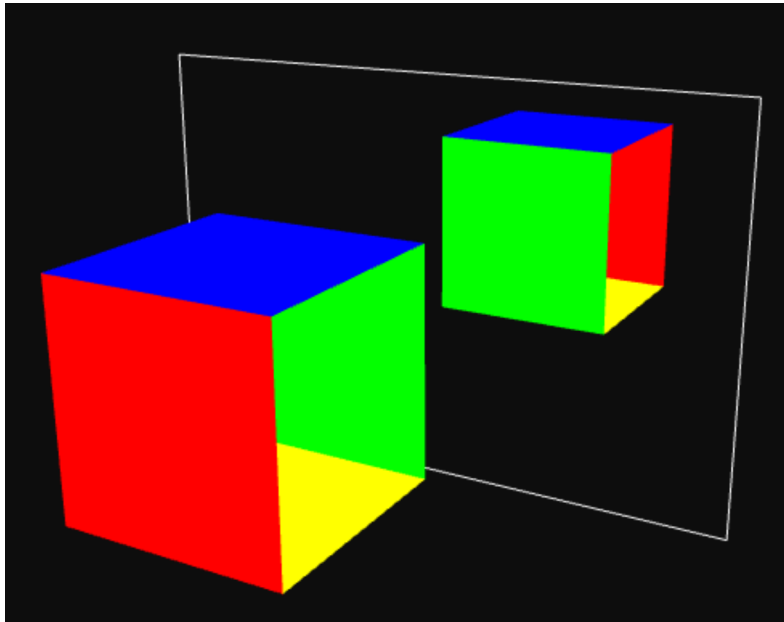
```
gl.stencilFunc( gl.EQUAL, 0x1, 0x1 );
```

3. Disegna oggetti dove lo stencil ha valori diversi da 1

```
gl.stencilFunc( gl.NOT_EQUAL, 0x1, 0x1 );
```

Lo Stencil Test avviene sui sample (per es. 4x) di un pixel.

Riflessioni planari (Stencil Mask)



Il cubo viene riflesso dallo specchio piano.
Si disegna il cubo due volte, la seconda volta lo si
"scala/riflette" di $[1,1,-1]$ per generare il simmetrico rispetto
allo specchio $Z=0$ (il riflesso attraverso lo specchio)



Codice di Esempio

L'effetto Riflessione Speculare con Stencil Buffer
è sperimentabile con il seguente codice:

Cartella: **HTML5_webgl_4**

Codice: `cube_stencil_mirror.html`



Resolve MSAA

Una tecnica molto semplice per fare Full Screen Anti-Aliasing (FSAA) è la **Super Sampling Anti-Aliasing** che consiste nel fare il rendering della scena ad una risoluzione maggiore rispetto a quella richiesta, quindi nel fare down-sampling alla risoluzione della viewport;

questa tecnica ha come inconveniente che è molto costosa; una sua semplificazione è nota come **MultiSample Anti-Aliasing (MSAA)**.

WebGL implementa l'**MSAA** che è attivo di default; per disattivarlo all'inizio si deve settare:

```
var context = canvas.getContext('webgl',{antialias:false});
```



Resolve MSAA

Il **Resolve** è l'operazione di chiusura che avviene **dopo** che tutte le 'Per-Fragment Operations' sono terminate per tutti gli oggetti della scena.

- **Cosa fa:** prende i 4 campioni calcolati e salvati nelle fasi precedenti e ne fa la media aritmetica per ottenere il colore finale del pixel (il "colore antialias")
- **Risultato:** un'immagine 2D standard pronta per essere mostrata a video o usata come texture

Il **Resolve MSAA** avviene prima del **Dithering** (o meglio, il dithering viene applicato durante la fase di scrittura dell'immagine finale risolta)

Sul colore "medio" appena calcolato, la GPU applica il micro-rumore del dithering per arrotondarlo correttamente agli 8-bit (o ai bit dello schermo) senza creare bande di colore.



Dithering

Il suo scopo è nascondere un difetto visivo chiamato **Color Banding** (sfumature a "gradini").

Questo accade perché i calcoli nello shader avvengono ad alta precisione (32-bit float), ma il monitor e il framebuffer spesso supportano solo 8-bit per canale (256 livelli di colore). Quando si ha una sfumatura molto lieve (per esempio le soft shadow per le ombre e penombre), la scheda video deve arrotondare i colori, creando strisce nette di colore diverso.

Il **Dithering** risolve il problema aggiungendo un **rumore impercettibile** (un pattern di pixel leggermente diversi) che "inganna" l'occhio umano, facendogli percepire una sfumatura fluida dove tecnicamente ci sono solo pochi colori disponibili.



Software Rendering

Se un computer non ha una scheda grafica dedicata (GPU), WebGL e GLSL continuano a funzionare grazie a un processo chiamato **Software Rendering** (o Rasterizzazione Software).

In assenza di una GPU, il lavoro che normalmente verrebbe svolto da migliaia di piccoli core grafici viene "emulato" dai pochi e potenti core della CPU.

Il sistema operativo o il browser utilizzano librerie specifiche (come **SwiftShader** di Google o **Mesa LLVMpipe** su Linux) per tradurre il codice GLSL in istruzioni x86/ARM che la CPU può eseguire. Sui sistemi Windows, WebGL usa una libreria chiamata **ANGLE**. Se non trova driver grafici validi (DirectX/OpenGL), ANGLE può passare al rendering software.



Software Rendering

Cosa succede in pratica:

- **Vertex Shader:** la CPU calcola le trasformazioni matematiche (matrici, posizioni) per ogni vertice in modo sequenziale o parallelo (usando istruzioni SIMD come SSE/AVX).
- **Rasterizzazione:** la CPU determina quali pixel dello schermo sono coperti dai triangoli della geometria.
- **Fragment Shader:** la CPU esegue il codice GLSL per ogni singolo pixel. Qui il sistema rallenta drasticamente: mentre una GPU esegue per milioni di pixel simultaneamente, la CPU deve fare i conti con molte meno unità di calcolo.

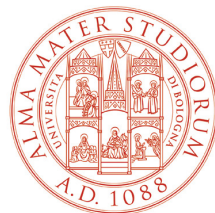
E' possibile dare un comando JavaScript per sapere se un codice utente sta usando il **Software Rendering**.



Applications

Real World Applications using WebGL

<https://manu.ninja/25-real-world-applications-using-webgl/>



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Giulio Casciola
Dip. di Matematica
giulio.casciola@unibo.it